Actividad 6: Taller Práctico de Repaso y Consolidación en C++

Proyecto REPASOGENERAL

Ian Quiroga, Santino Laudati y Dante Iglesias

EEST N.º 1 “Eduardo Ader” - Vicente López

Laboratorio de Programación - 5.º Año 3.º A-B

Profesor: Mansilla Muñoz York Elías

Yamil De Jesus Ganduglia

2026

Fecha de entrega: 4 de noviembre de 2026

Repositorio de GitHub: https://github.com/pachusUwu/repasogeneral.git

Control de versiones del documento

Versión	Fecha	Descripción	Autores
v1.0.0	04/11/2026	Baseline Aprobado	I. Quiroga, S. Laudati, D. Iglesias

Introducción

Esta actividad tuvo como fin consolidar los tres pilares de bajo nivel trabajados en la primera mitad del año: la **recursividad** y la pila de llamadas (*Stack*), la **búsqueda secuencial** en arreglos de memoria contigua y el **intercambio físico de valores mediante punteros**. Los tres retos se integraron en una única suite (`src/main.cpp`) que se compiló y ejecutó en la terminal de PowerShell de VS Code.

Para resolverla se aplicó el Método 5R (Registrar, Reducir, Responder, Reflexionar y Revisar). Además, se configuró la estrategia anticaídas *Dual-Remote*, para que cada `git push` envíe el código a GitHub y a GitLab al mismo tiempo. El documento incluye la tabla de control de revisiones (DocVer) en la portada con la versión `v1.0.0 Baseline Aprobado`.

Desarrollo

Integrantes y tareas realizadas

Tabla 1

Distribución de tareas del grupo

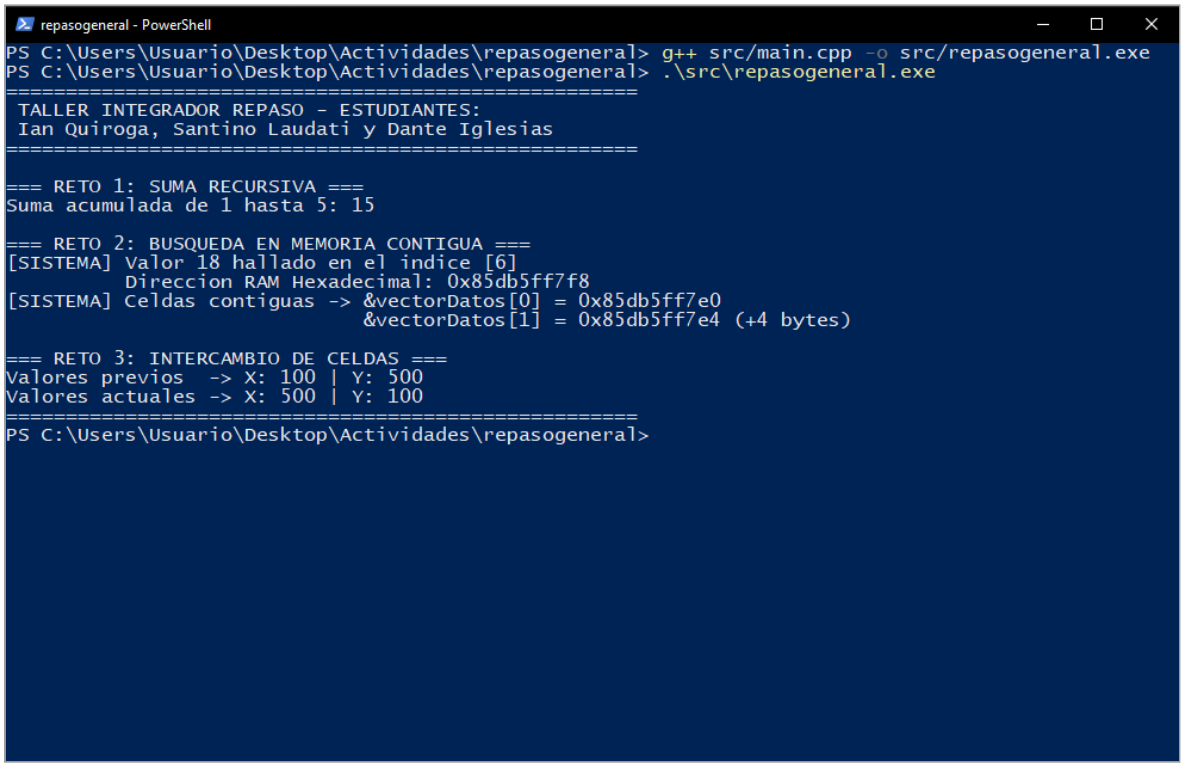
Integrante	Tareas realizadas
Ian Quiroga	R1 y R2: transcripción, compilación y ejecución de la suite; identificación del núcleo de cada reto; agregado de la comprobación de celdas contiguas.
Santino Laudati	R3 y R4: respuestas del cuestionario, reflexión sobre el P.I.A., redacción del informe APA v7 y del manual del usuario.
Dante Iglesias	R5: configuración Dual-Remote (GitHub + GitLab), `docs/CHANGELOG.md`, traza de memoria, capturas, README y manual del programador.

R1 – Registrar: compilación y ejecución

```
g++ src/main.cpp -o src/repasogeneral.exe
.\src\repasogeneral.exe
```

Figura 1

Ejecución de la suite de repaso con los tres retos



Nota. La suma recursiva de 1 a 5 dio 15; el valor 18 se halló en el índice [6] (dirección 0x85db5ff7f8); X e Y se intercambiaron (500 y 100).

R2 – Reducir: el núcleo de cada reto

Tabla 2

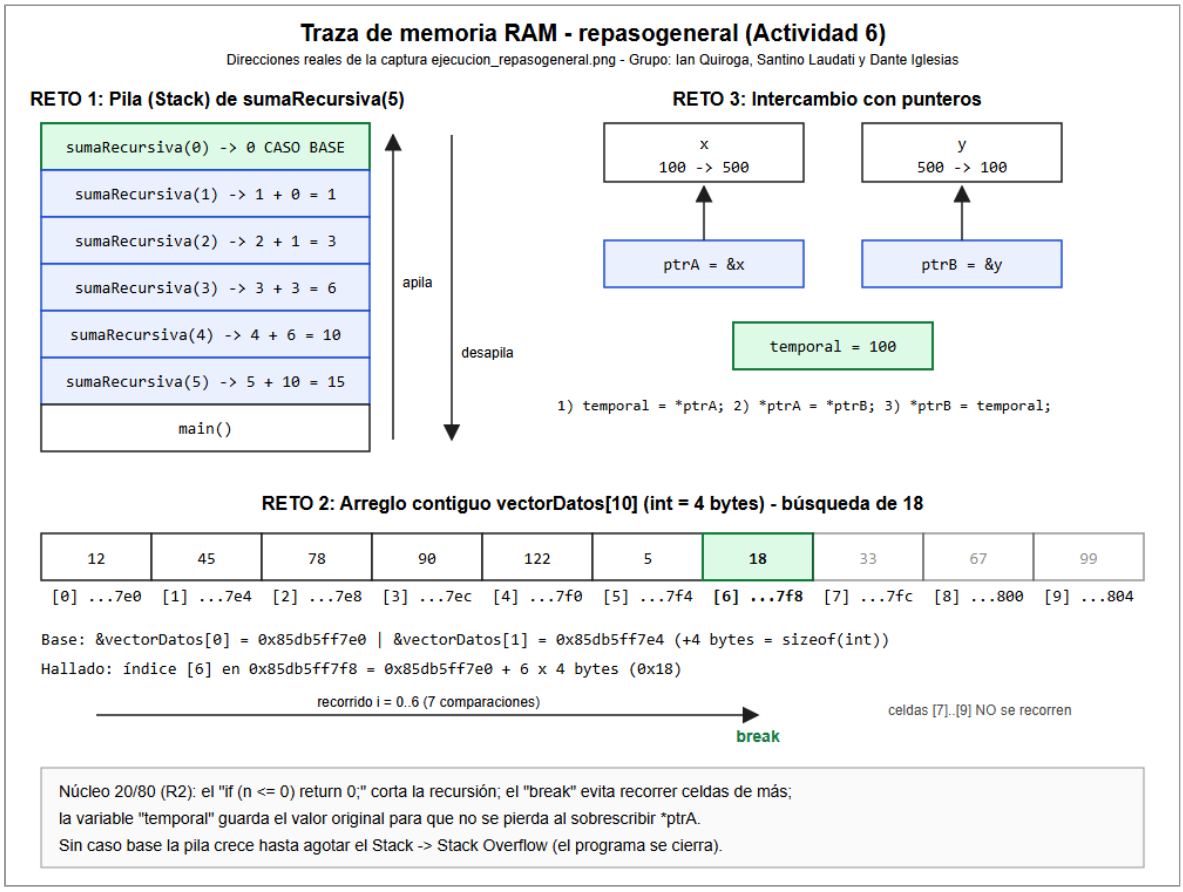
El 20 % del código que produce el 80 % del resultado

Reto	Línea núcleo	Por qué es la clave
1. Recursividad	`if (n <= 0) return 0;`	Caso base: corta la cadena de llamadas y evita el Stack Overflow.
2. Búsqueda	`break;`	Interrumpe el `for` apenas encuentra el valor; no recorre celdas innecesarias.
3. Intercambio	`int temporal = *ptrA;`	Resguarda el valor original de X antes de sobrescribirlo.

En el reto 2 el grupo agregó la impresión de `&vectorDatos[0]` y `&vectorDatos[1]`: en la ejecución se observan las direcciones `0x85db5ff7e0` y `0x85db5ff7e4`, separadas exactamente por 4 bytes, el tamaño de un `int`.

Figura 2

Traza de memoria RAM de los tres retos (pila de llamadas, arreglo contiguo e intercambio con punteros)



R3 – Responder: cuestionario técnico de control

1. Caso base. Si una función recursiva no tiene caso base, se llama a sí misma indefinidamente. Cada llamada reserva un nuevo bloque (*stack frame*) en la pila con sus variables locales y su dirección de retorno; como ninguna llamada termina, la pila crece hasta superar el espacio asignado por el sistema operativo (alrededor de 1 MB en Windows) y se produce un **desbordamiento de pila (Stack Overflow)**: el programa se cierra abruptamente con un error de acceso a memoria.

2. Eficiencia. La búsqueda secuencial tiene complejidad O(n): en el peor caso compara las n celdas. La instrucción `break` corta el ciclo en cuanto encuentra el valor, de modo que el microprocesador no ejecuta comparaciones, saltos ni accesos a memoria innecesarios. En nuestro arreglo de 10 elementos, el 18 se encontró en la séptima comparación y se evitaron las 3 restantes; en arreglos de millones de datos el ahorro es enorme.

3. Direccionamiento. Basta imprimir con el operador `&` la dirección de elementos consecutivos (`&vectorDatos[0]`, `&vectorDatos[1]`, ...) y restar: si cada dirección es la anterior más `sizeof(int)` (4 bytes), las celdas son contiguas. En nuestra ejecución, `0x85db5ff7e4 − 0x85db5ff7e0 = 4`, y el índice 6 está en `0x85db5ff7e0 + 6 × 4 = 0x85db5ff7f8`, tal como mostró el programa.

4. Variables temporales. La variable `temporal` funciona como un casillero auxiliar. Al ejecutar `*ptrA = *ptrB;` el valor original de X (100) se sobrescribe y se perdería; por eso antes se guarda en `temporal`, y al final se asigna `*ptrB = temporal;`. Sin ella, ambas variables terminarían con el mismo valor (500).

5. Autoevaluación. El reto que más nos costó comprender fue el de recursividad, porque cuesta imaginar que la función "queda en pausa" esperando el resultado de la llamada siguiente. Lo resolvimos dibujando la pila de llamadas (ver la traza de memoria): primero se apilan `sumaRecursiva(5)` hasta `sumaRecursiva(0)`, y luego, al llegar al caso base, se van resolviendo de arriba hacia abajo (0, 1, 3, 6, 10, 15).

R4 – Reflexionar: conexión con el Proyecto Integrador

Estas técnicas son aplicables al prototipo del P.I.A. La búsqueda secuencial con `break` sirve para recorrer un buffer de lecturas de un sensor y detectar el primer valor que supere un umbral de alarma. El intercambio con punteros es la base de los algoritmos de ordenamiento (por ejemplo, para ordenar lecturas de menor a mayor), y el pasaje por

dirección permite que una función actualice directamente las variables de estado del dispositivo. La recursividad, en cambio, debe usarse con cuidado en microcontroladores, cuya pila es muy pequeña.

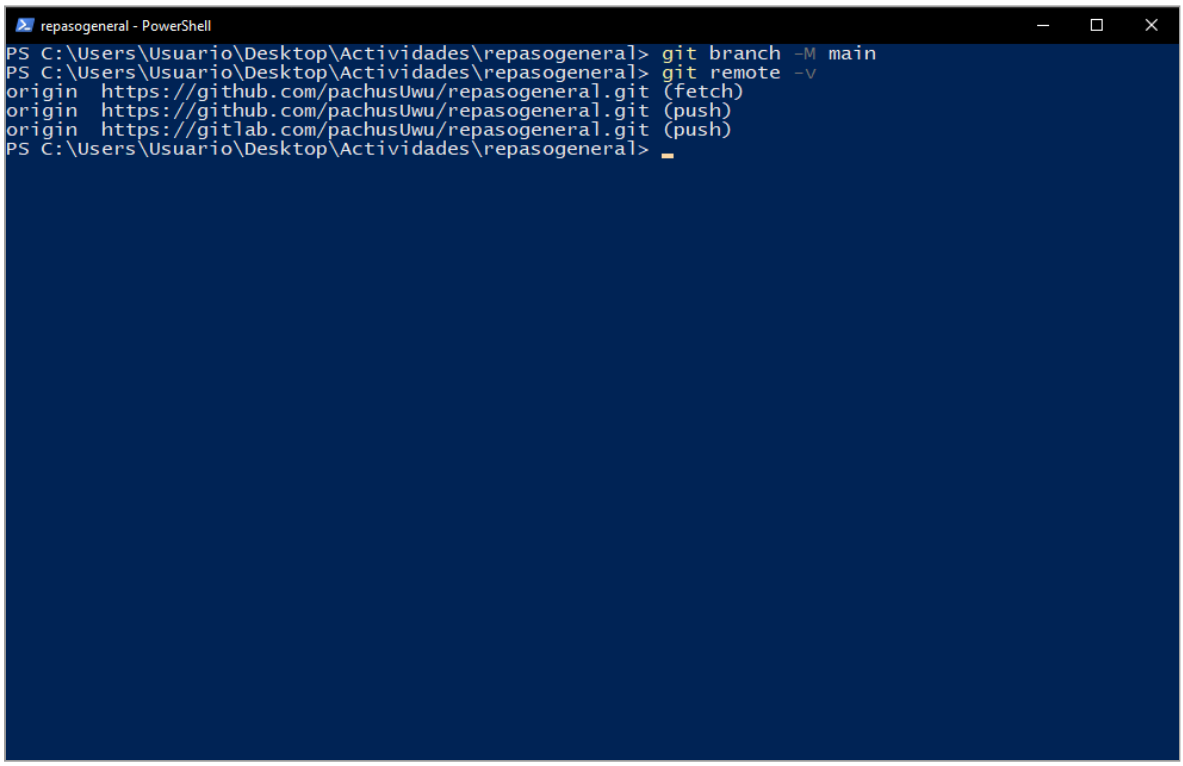
R5 – Revisar: estrategia Dual-Remote

Se configuró el remoto `origin` con una dirección de lectura (*fetch*) en GitHub y dos direcciones de envío (*push*): GitHub como principal y GitLab como respaldo de emergencia.

```
git remote set-url origin https://github.com/pachusUwu/repasogeneral.git
git remote set-url --add --push origin
https://github.com/pachusUwu/repasogeneral.git
git remote set-url --add --push origin
https://gitlab.com/pachusUwu/repasogeneral.git
git remote -v
git branch -M main
```

Figura 3

Salida de git remote -v: una dirección de fetch y dos de push (GitHub y GitLab)



A partir de esta configuración, un único `git push` envía los cambios a ambos servidores. La bitácora de versiones quedó registrada en `docs/CHANGELOG.md`.

Conclusión

El taller nos permitió unir en un solo programa los conceptos de bajo nivel vistos durante el cuatrimestre. Comprobamos con direcciones reales que un arreglo ocupa celdas contiguas de 4 bytes, entendimos por qué el caso base es obligatorio en una función recursiva y vimos cómo una variable temporal evita perder datos al intercambiar valores mediante punteros.

El Método 5R nos ayudó a no limitarnos a copiar el código, sino a identificar las líneas clave y explicarlas. Por último, la estrategia Dual-Remote nos enseñó una práctica profesional sencilla: tener siempre un respaldo del código en un segundo servidor, para no depender de un único proveedor cuando se acerque la Expotécnica.

Referencias

cplusplus.com. (s.f.). *Arrays*. https://cplusplus.com/doc/tutorial/arrays/

Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/es/v2

GeeksforGeeks. (2025). *Linear search algorithm*. https://www.geeksforgeeks.org/dsa/linear-search/

Git. (s.f.). *git-remote documentation*. https://git-scm.com/docs/git-remote

Mansilla Muñoz, Y. E. (2026). *Actividad 6: Taller práctico de repaso y consolidación en C++* [Material de clase]. E.E.S.T. N.° 1 "Eduardo Ader".

Microsoft. (2024). *Matrices (C++)*. Microsoft Learn. https://learn.microsoft.com/es-es/cpp/cpp/arrays-cpp

Programiz. (s.f.). *C++ recursion*. https://www.programiz.com/cpp-programming/recursion

Quiroga, I., Laudati, S., & Iglesias, D. (2026a). *repasogeneral* [Repositorio de código fuente]. GitHub. https://github.com/pachusUwu/repasogeneral

Quiroga, I., Laudati, S., & Iglesias, D. (2026b). *repasogeneral* [Repositorio espejo]. GitLab. https://gitlab.com/pachusUwu/repasogeneral